
PREDICTION OF EPILEPTIC SEIZURES USING EEG SIGNALS

Signal and Systems Project - Phase 2



Instructor : Prof. Hamid Aghajan

Moein Abdollahi Sarvi - 401100885

Spring 2024

Contents

1	Load Database	3
2	Calculating Power Spectral Density (PSD)	3
3	Calculating Shannon Entropy	5
4	Feature Extraction	7
5	Feature Selection	9
6	Support Vector Machine (SVM) Classifier	11
7	K-Nearest Neighbor (KNN) Classifier	14

1 Load Database

In this study, I used the `edfread` function in MATLAB to load the data properly. The MATLAB code for loading the data is shown below:

```
1 data0 = edfread("chb01_02.edf")
2 data1 = edfread("chb01_03.edf")
3 data2 = edfread("chb01_04.edf")
4 data3 = edfread("chb01_15.edf")
5 data4 = edfread("chb01_16.edf")
6 data5 = edfread("chb01_18.edf")
7 data6 = edfread("chb01_26.edf")
```

2 Calculating Power Spectral Density (PSD)

Power Spectral Density (PSD) characterizes how the power of a signal or time series is distributed across different frequency components. Mathematically, the PSD $S_{xx}(f)$ of a signal $x(t)$ can be defined as the Fourier transform of the autocorrelation function $R_{xx}(\tau)$:

$$S_{xx}(f) = \int_{-\infty}^{\infty} R_{xx}(\tau) e^{-j2\pi f\tau} d\tau \quad (1)$$

Alternatively, it can be expressed in terms of the signal's Fourier transform $X(f)$ as:

$$S_{xx}(f) = \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E} [|X_T(f)|^2] \quad (2)$$

where $X_T(f)$ is the finite-time Fourier transform of $x(t)$ over a period T , and $\mathbb{E}$ denotes the expected value. The PSD provides insights into the signal's frequency content and is essential for investigating different EEG signals.

In MATLAB the PSD can be calculated using `periodogram` function. following lines of code shows how I calculated PSD of the first epoch of first channel of "chb01-03.edf".

```
1 signal1_firstChannel = reshape([data1.SignalLabel1_FP1_F7{:}], [], 1);
2
3 epoch_lenght = 16 * 256;
4 firstEpoch1 = signal1_firstChannel(1:16*256);
5
6 window = hamming(16*256);
7 noverlap = floor(16*256 / 5);
8 nfft = 2^nextpow2(16*256);
9 [psd, freq] = periodogram(firstEpoch1, window, nfft, 256, 'psd');
```

PSD calculated for first channel in first epoch is illustrated in Figure 1.

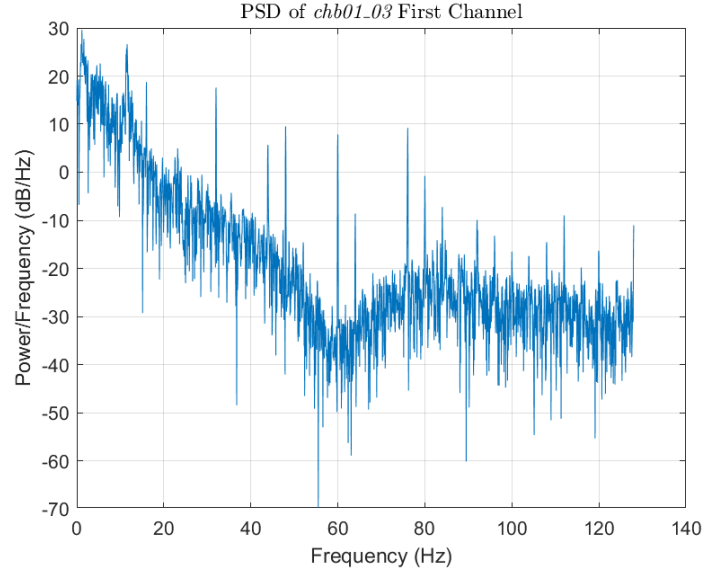


Figure 1: PSD of first epoch (for Channel 1: FP1-F7)

Also PSD in first epoch can be calculated for summation of all channels:

```

1 channelLabels = { ...
2     'SignalLabel1_FP1_F7', 'SignalLabel12_F7_T7', 'SignalLabel13_T7_P7', '
   SignalLabel14_P7_O1', ...
3     'SignalLabel15_FP1_F3', 'SignalLabel16_F3_C3', 'SignalLabel17_C3_P3', '
   SignalLabel18_P3_O1', ...
4     'SignalLabel19_FP2_F4', 'SignalLabel110_F4_C4', 'SignalLabel111_C4_P4', '
   SignalLabel112_P4_O2', ...
5     'SignalLabel13_FP2_F8', 'SignalLabel114_F8_T8', 'SignalLabel115_T8_P8', '
   SignalLabel116_P8_O2', ...
6     'SignalLabel17_FZ_CZ', 'SignalLabel118_CZ_PZ', 'SignalLabel19_P7_T7', '
   SignalLabel20_T7_FT9', ...
7     'SignalLabel21_FT9_FT10', 'SignalLabel22_FT10_T8', 'SignalLabel23_T8_P8'
   ...
8 };
9
10 fs = 256;
11 epoch_length = 16 * fs;
12
13 window = hamming(epoch_length);
14 noverlap = floor(epoch_length / 5);
15 nfft = 2^nextpow2(epoch_length);
16
17 signal1_allChannels = reshape([data1.(channelLabels{1}){:}], [], 1);
18
19 for i = 2:length(channelLabels)
20     signal1_allChannels = signal1_allChannels + reshape([data1.(channelLabels
   {i}){:}], [], 1);
21 end
22
23 firstEpoch = signal1_allChannels(1:epoch_length);
24 [psd, freq] = periodogram(firstEpoch, window, nfft, fs, 'psd');

```

the plot is shown in Figure2.

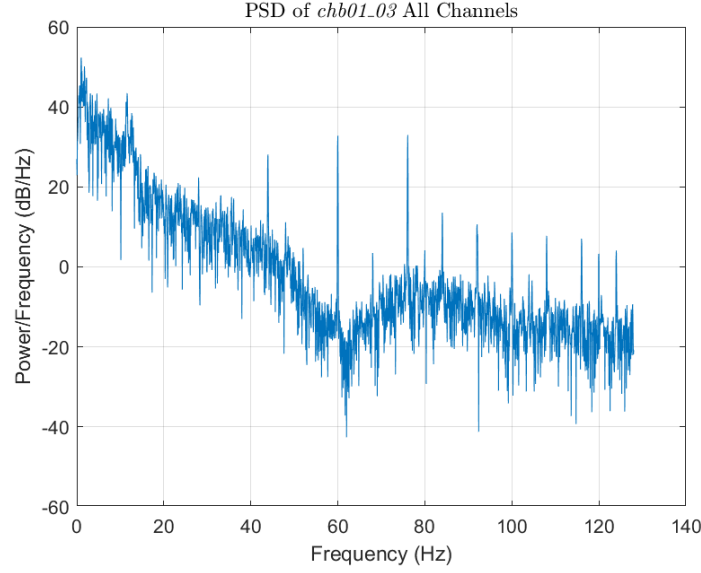


Figure 2: PSD of first epoch(for All Channels)

3 Calculating Shannon Entropy

Shannon entropy measures the uncertainty or randomness in a set of data, quantifying the expected amount of information produced by a stochastic source. In signal processing, it is often used to analyze the complexity of signals by treating Power Spectral Density (PSD) values as probability distributions. The formula for Shannon entropy is:

$$H(X) = - \sum_{i=1}^n p(x_i) \log p(x_i)$$

where $p(x_i)$ represents the probability of occurrence of each value x_i . Higher entropy indicates more complexity and disorder within the signal, reflecting a greater amount of unpredictability. Shannon entropy can also be applied in various fields such as data compression, cryptography, and machine learning, where it is used to optimize information encoding and enhance data security. Additionally, it serves as a fundamental tool in information theory, providing insights into the efficiency and capacity of communication systems.

The provided MATLAB code below calculates the Shannon entropy of a signal segment. Firstly, it defines the sampling rate, epoch duration, and the number of bins for the histogram. The signal data is then extracted and reshaped into a column vector. The code selects the first epoch of the signal based on the defined duration and computes its histogram to obtain frequency counts. These counts are converted into probabilities, excluding zeros, and the Shannon entropy is calculated using its formula. The calculation is for one channel of EEG data and can be expanded for all channels just like PSD.

```
1 samplingRate = 256;
2 epochDuration = 16;
3 numBins = 256;
```

```

4
5 rawSignal = cell2mat(data1.SignalLabel1_FP1_F7);
6 signalVector = rawSignal(:);
7
8 epochSampleCount = epochDuration * samplingRate;
9 epochSignal = signalVector(1:epochSampleCount);
10 [frequencyCounts, ~] = histcounts(epochSignal, numBins);
11
12 signalProbabilities = frequencyCounts / sum(frequencyCounts);
13 signalProbabilities(signalProbabilities == 0) = [];
14
15 shannonEntropy = -sum(signalProbabilities .* log2(signalProbabilities));
16 fprintf('Shannon entropy of the first %d-second epoch: %f\n', epochDuration,
    shannonEntropy);

```

Finally, the calculated entropy of the first epoch is 7.000565.

we also can calculate entropy for all epochs with same procedure. Below there is a MATLAB code for calculating Shannon entropy within all epochs.

```

1 flattenedData = cell2mat(data1.SignalLabel1_FP1_F7);
2 flattenedData = flattenedData(:);
3
4 samplesPerEpoch = 16 * 256;
5 totalEpochs = floor(length(flattenedData) / samplesPerEpoch);
6
7 shannonEntropy = zeros(1, totalEpochs);
8
9 for epochIndex = 1:totalEpochs
10     startIdx = (epochIndex - 1) * samplesPerEpoch + 1;
11     endIdx = epochIndex * samplesPerEpoch;
12     epochData = flattenedData(startIdx:endIdx);
13
14     binCount = 256;
15     histogramCounts = histcounts(epochData, binCount);
16     probDist = histogramCounts / sum(histogramCounts);
17     probDist(probDist == 0) = [];
18
19     shannonEntropy(epochIndex) = -sum(probDist .* log2(probDist));
20 end
21
22 figure;
23 plot(1:totalEpochs, shannonEntropy, '-o', 'Color', [0 0.5 0.5], '
    MarkerEdgeColor', 'k', 'MarkerFaceColor', 'y');
24 xlabel('Epoch Number');
25 ylabel('Shannon Entropy');
26 grid on;

```

The provided MATLAB script computes the Shannon entropy of EEG data for the channel FP1-F7, segmented into 16-second epochs. By flattening the channel data and dividing it into epochs, the script calculates the probability distribution of signal values within each epoch using histogram counts. Shannon entropy is then computed for each epoch to measure the uncertainty or randomness of the signal. The results are plotted, showing the entropy

values across epochs, with a continuous line for visual clarity. This analysis helps in understanding the variability and complexity of brain activity over time in the specified channel. Result is shown in Figure3.

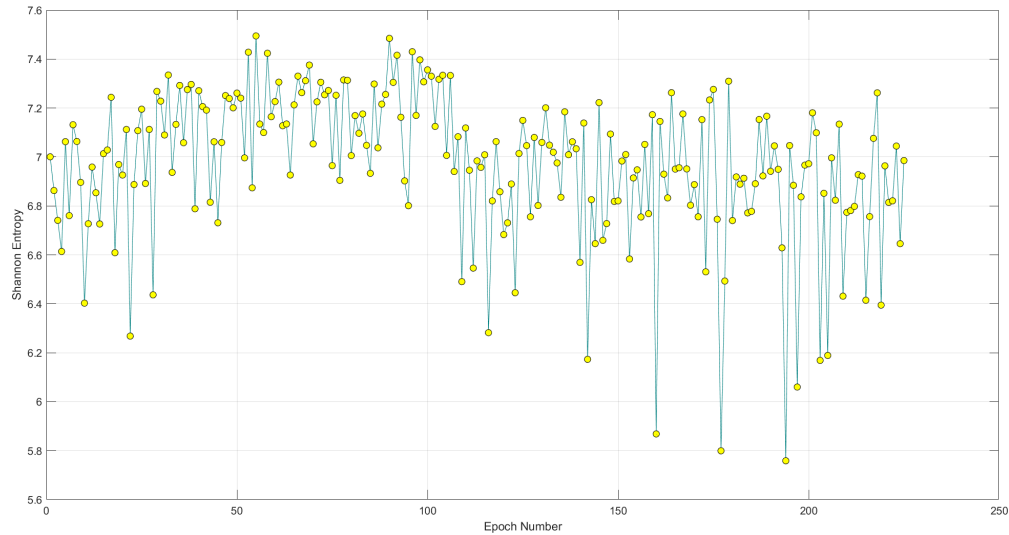


Figure 3: Shannon Entropy For All Epochs of First Channel

4 Feature Extraction

In this section, the task is to extract features from EEG signals, specifically focusing on the 10-minute intervals leading up to a seizure. The EEG data is segmented into 16-second epochs using a moving window approach. For each epoch, the Fast Fourier Transform (FFT) is computed to obtain the Power Spectral Density (PSD), which is then normalized. Additionally, Shannon entropy is calculated to measure the complexity of the signal. Other statistical measures such as average, standard deviation, minimum, and maximum are also computed to provide a comprehensive analysis of the EEG signal characteristics in each epoch.

```

1  clc;
2  data = data1.SignalLabel1_FP1_F7;
3  EEG = cell2mat(data);
4  EEG = EEG(:);
5
6  Fs = 256;
7  epochSec = 16;
8  epochSamp = epochSec * Fs;
9  totalSec = 10 * 60;
10 totalSamp = totalSec * Fs;
11 numEpochs = floor(totalSamp / epochSamp);
12
13 epochLabels = randi([0, 1], numEpochs, 1);
14
15 features = zeros(numEpochs, 5);
16

```

```

17 for idx = 1:numEpochs
18     startIdx = (idx - 1) * epochSamp + 1;
19     endIdx = idx * epochSamp;
20     epochData = EEG(startIdx:endIdx);
21
22     fftData = fft(epochData);
23
24     psdData = (1/(Fs * epochSamp)) * abs(fftData).^2;
25     psdData = psdData(1:floor(epochSamp / 2) + 1);
26     psdData(2:end-1) = 2 * psdData(2:end-1);
27
28     psdData = psdData / sum(psdData);
29
30     psdData(psdData == 0) = [];
31     entropy = -sum(psdData .* log2(psdData));
32
33     meanVal = mean(epochData);
34     stdVal = std(epochData);
35     minVal = min(epochData);
36     maxVal = max(epochData);
37
38     features(idx, :) = [meanVal, stdVal, minVal, maxVal, entropy];
39 end
40
41 featureTable = array2table(features, 'VariableNames', {'Mean', 'StdDev', 'Min',
42     'Max', 'ShannonEntropy'});
43 disp(featureTable);

```

The provided MATLAB code extracts features from EEG data for analysis. Feature extraction is done for the first data and for all epochs. Same as previous parameters that have been calculated in this project, these features can easily be extracted from other EEG files too. The continuous EEG signal is segmented into multiple 16-second epochs, with the number of epochs determined by the total data duration and sampling frequency. For each epoch, the Fast Fourier Transform (FFT) is computed to convert the signal from the time domain to the frequency domain, and the Power Spectral Density (PSD) is calculated to represent the power distribution over the frequency components. The PSD is normalized, and Shannon entropy is computed to quantify signal complexity. Additionally, statistical features such as mean, standard deviation, minimum, and maximum values are extracted from the time-domain signal of each epoch. These features are compiled into a matrix and converted into a table for further analysis. Final table is shown in Figure4

Mean	StdDev	Min	Max	ShannonEntropy
0.33692	29.854	-133.43	96.703	6.8968
0.36554	38.72	-152.19	171.33	6.6573
-0.33129	53.831	-245.57	220.17	6.7889
0.39206	27.49	-163.52	92.015	6.956
0.082799	25.171	-104.13	77.949	6.6606
0.17199	18.387	-73.26	84.591	7.4084
-0.20232	22.325	-81.465	70.525	7.1136
0.54516	20.995	-74.042	72.869	6.905
0.24907	38.005	-172.89	140.07	6.5791
0.17981	40.261	-178.36	237.36	6.1913
0.73346	34.011	-157.66	133.82	6.4448
-0.57645	27.659	-138.51	77.558	7.0841
0.39349	31.108	-155.7	97.485	6.4582
-0.039683	25.066	-138.51	90.842	7.0481
-0.078888	24.488	-99.048	88.498	6.9115
0.82561	24.939	-100.61	88.498	6.8504
-0.27186	32.476	-99.048	114.68	6.7283
-0.3084	44.139	-213.14	190.87	6.1996
0.49031	41.221	-161.56	166.25	6.1902
0.70761	31.09	-124.44	123.66	6.7405
-0.67203	38.342	-140.46	131.09	6.6686
0.38652	49.085	-267.06	297.53	6.2276
0.71562	30.099	-126.79	122.88	6.7348
0.10522	33.402	-145.15	92.796	6.6823
-1.0687	38.944	-145.15	118.58	6.3875
2.001	41.8	-136.17	199.46	6.4562
-0.44051	38.812	-148.67	131.87	6.5332
0.017361	55.862	-343.25	236.97	6.3067
0.52656	35.56	-97.485	130.31	6.0644
-0.16674	37.815	-139.29	112.72	6.2921
-0.11294	40.093	-141.25	147.11	6.0556
0.41896	40.22	-128.35	117.02	6.2633
0.55384	44.716	-217.44	134.99	6.2057
-0.34646	40.853	-154.92	133.82	6.3213
-0.3228	43.657	-172.11	97.485	6.1032
0.71305	47.989	-180.71	176.41	6.3019
-0.37155	38.498	-133.82	113.11	6.0428

Figure 4: Features Extracted From All Epochs of First Data

5 Feature Selection

Feature selection is a critical step in machine learning to identify and retain significant variables that contribute most to the prediction target, enhancing model performance and interpretability. Using a t-test for feature selection involves applying a statistical hypothesis test to determine if the mean of a feature differs significantly from a hypothesized value, typically zero. The t-test computes a t-statistic,

$$t = \frac{\bar{X} - \mu_0}{s/\sqrt{n}}$$

where $\bar{X}$ is the sample mean, μ_0 is the hypothesized mean, s is the standard deviation, and n is the sample size. A low p-value indicates that the null hypothesis (the feature's mean equals the hypothesized mean) can be rejected, suggesting that the feature has a statistically significant impact. This method helps in filtering out irrelevant or redundant features, thereby simplifying models and reducing overfitting. However, it assumes normality of the data and independence among features, which may not always hold in practice.

```

1  clc;
2  mu = 0;
3  numFeat = size(features, 2);
4  pVals = zeros(1, numFeat);
5  tVals = zeros(1, numFeat);
6
7  for i = 1:numFeat
8      [~, p, ~, stats] = ttest(features(:, i), mu, 'Alpha', 0.001);
9      pVals(i) = p;
10     tVals(i) = stats.tstat;
11 end
12
13 selFeat = features(:, pVals < 0.001);
14
15 disp('P-values:');
16 disp(pVals);
17 disp('T-statistics:');
18 disp(tVals);
19 disp('Selected features:');
20 disp(selFeat);

```

selected features are shown in Figure5

T-statistics:				
1.6742	24.2737	-17.1070	15.4164	114.6359
Selected features:				
29.8544	-133.4310	96.7033	6.8968	
38.7198	-152.1856	171.3309	6.6573	
53.8308	-245.5678	220.1709	6.7889	
27.4901	-163.5165	92.0147	6.9560	
25.1707	-104.1270	77.9487	6.6606	
18.3870	-73.2601	84.5910	7.4084	
22.3246	-81.4652	70.5250	7.1136	
20.9946	-74.0415	72.8694	6.9050	
38.0055	-172.8938	140.0733	6.5791	
40.2614	-178.3639	237.3626	6.1913	
34.0112	-157.6557	133.8217	6.4448	
27.6595	-138.5104	77.5580	7.0841	
31.1076	-155.7021	97.4847	6.4582	
25.0662	-138.5104	90.8425	7.0481	
24.4883	-99.0476	88.4982	6.9115	
24.9386	-100.6105	88.4982	6.8504	
32.4759	-99.0476	114.6764	6.7283	
44.1394	-213.1380	190.8669	6.1996	
41.2213	-161.5629	166.2515	6.1902	
31.0896	-124.4444	123.6630	6.7405	
38.3420	-140.4640	131.0867	6.6686	
49.0854	-267.0574	297.5336	6.2276	
30.0989	-126.7888	122.8816	6.7348	
33.4025	-145.1526	92.7961	6.6823	
38.9438	-145.1526	118.5836	6.3875	
41.7999	-136.1661	199.4628	6.4562	
38.8123	-148.6691	131.8681	6.5332	
55.8618	-343.2479	236.9719	6.3067	
35.5603	-97.4847	130.3053	6.0644	
37.8148	-139.2918	112.7228	6.2921	
40.0934	-141.2454	147.1062	6.0556	
40.2195	-128.3516	117.0208	6.2633	
44.7161	-217.4359	134.9939	6.2057	
40.8534	-154.9206	133.8217	6.3213	
43.6565	-172.1123	97.4847	6.1032	
47.9894	-180.7082	176.4103	6.3019	
38.4980	-133.8217	113.1136	6.0428	

Figure 5: Selected Features Based on The Test

6 Support Vector Machine (SVM) Classifier

A Support Vector Machine (SVM) classifier is a powerful supervised learning algorithm used for classification tasks by finding the optimal hyperplane that separates different classes in the feature space. The hyperplane, which is an $(n - 1)$ -dimensional subspace (e.g., a line in 2D, a plane in 3D), is chosen to maximize the margin—the distance between the hyperplane and the nearest data points from each class, known as support vectors. This maximized margin enhances the model’s generalization ability to unseen data. SVMs can use a linear kernel for linearly separable data or non-linear kernels like polynomial or radial basis function (RBF) for more complex distributions. They are particularly effective in high-dimensional spaces and robust to overfitting, especially when the number of features exceeds the number of samples.

The following MATLAB code trains and evaluates a Support Vector Machine (SVM) classifier with a linear kernel. The code performs the following steps:

1. Partition the dataset into training (70%) and testing (30%) sets using `cvpartition`.
2. Extract the training and testing features and labels using the indices from `cvpartition`.
3. Train the SVM classifier on the training data using `fitcsvm` with a linear kernel.
4. Predict the labels of the test data using the trained SVM model.
5. Generate the confusion matrix using `confusionmat`.
6. Calculate performance metrics: accuracy, sensitivity, specificity, and latency.

The performance metrics are defined as follows:

- **Accuracy** is the proportion of correctly classified instances and is calculated as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Sensitivity (Recall)** is the proportion of true positives correctly identified by the classifier and is calculated as:

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

- **Specificity** is the proportion of true negatives correctly identified by the classifier and is calculated as:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

- **Latency** is the average time taken to make a single prediction and is calculated by measuring the total prediction time for the test set and dividing it by the number of test samples.

Where:

- TP is the number of true positives.

- TN is the number of true negatives.
- FP is the number of false positives.
- FN is the number of false negatives.

```

1  clc;
2  cv = cvpartition(labels, 'HoldOut', 0.3);
3  trainIdx = training(cv);
4  testIdx = test(cv);
5
6  X_train = features(trainIdx, :);
7  y_train = labels(trainIdx);
8  X_test = features(testIdx, :);
9  y_test = labels(testIdx);
10
11 SVMModel = fitcsvm(X_train, y_train, 'KernelFunction', 'linear');
12
13 y_pred = predict(SVMModel, X_test);
14
15 confMat = confusionmat(y_test, y_pred);
16
17 tp = confMat(1,1);
18 tn = confMat(2,2);
19 fp = confMat(2,1);
20 fn = confMat(1,2);
21
22 accuracy = (tp + tn) / sum(confMat(:));
23 sensitivity = tp / (tp + fn);
24 specificity = tn / (tn + fp);
25
26 numTestSamples = size(X_test, 1);
27 tic;
28 for i = 1:numTestSamples
29     predict(SVMModel, X_test(i, :));
30 end
31 latency = toc / numTestSamples;
32
33 disp('Confusion Matrix:');
34 disp(confMat);
35 disp(['Accuracy: ', num2str(accuracy)]);
36 disp(['Sensitivity: ', num2str(sensitivity)]);
37 disp(['Specificity: ', num2str(specificity)]);
38 disp(['Latency (average prediction time): ', num2str(latency), ' seconds']);

```

To ensure reproducibility of results when running the SVM classifier code, it is essential to set a random seed before partitioning the data. The 'cvpartition' function with the 'HoldOut' method randomly selects training and test sets each time the code runs, leading to variations in performance metrics such as accuracy, sensitivity, specificity, and latency. By setting a random seed using the 'rng' function, we can control the randomness in the data partitioning process, ensuring consistent results across different runs. Below is the updated MATLAB code with a fixed random seed:

```

1  clc;
2  rng(1);
3  cv = cvpartition(labels, 'HoldOut', 0.3);
4  trainIdx = training(cv);
5  testIdx = test(cv);
6
7  X_train = features(trainIdx, :);
8  y_train = labels(trainIdx);
9  X_test = features(testIdx, :);
10 y_test = labels(testIdx);
11
12 SVMModel = fitcsvm(X_train, y_train, 'KernelFunction', 'linear');
13
14 y_pred = predict(SVMModel, X_test);
15
16 confMat = confusionmat(y_test, y_pred);
17
18 tp = confMat(1,1);
19 tn = confMat(2,2);
20 fp = confMat(2,1);
21 fn = confMat(1,2);
22 accuracy = (tp + tn) / sum(confMat(:));
23 sensitivity = tp / (tp + fn);
24 specificity = tn / (tn + fp);
25 numTestSamples = size(X_test, 1);
26 tic;
27 for i = 1:numTestSamples
28     predict(SVMModel, X_test(i, :));
29 end
30 latency = toc / numTestSamples;
31
32 disp('Confusion Matrix:');
33 disp(confMat);
34 disp(['Accuracy: ', num2str(accuracy)]);
35 disp(['Sensitivity: ', num2str(sensitivity)]);
36 disp(['Specificity: ', num2str(specificity)]);
37 disp(['Latency (average prediction time): ', num2str(latency), ' seconds']);

```

The inclusion of `rng(1);` ensures that the random number generator produces the same sequence of random numbers each time the code is executed, resulting in consistent partitioning of the dataset into training and testing sets, and consequently, consistent performance metrics. The seed value (e.g., 1) can be changed to any other integer as needed.

```

Confusion Matrix:
     4     3
     2     2

Accuracy: 0.54545
Sensitivity: 0.57143
Specificity: 0.5
Latency (average prediction time): 0.00034791 seconds

```

Figure 6: SVM Results

7 K-Nearest Neighbor (KNN) Classifier

The following MATLAB code trains and evaluates a K-Nearest Neighbor (KNN) classifier using the Minkowski distance metric. The process includes partitioning the data, training the classifier, predicting labels, and evaluating performance metrics.

1. **Data Partitioning:** The dataset is split into training (70%) and testing (30%) sets using the `cvpartition` function.
2. **Feature and Label Extraction:** Training and testing features and labels are extracted based on the partition indices.
3. **Training the KNN Classifier:** The KNN classifier is trained on the training data using the Minkowski distance metric with 5 neighbors, implemented by the `fitcknn` function.
4. **Prediction:** The trained KNN model predicts the labels of the test data using the `predict` function.
5. **Confusion Matrix Generation:** A confusion matrix is generated using the `confusionmat` function to compare the predicted and actual test labels.
6. **Performance Metrics Calculation:**

- **Accuracy:** The proportion of correctly classified instances, calculated as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Sensitivity (Recall):** The proportion of true positives correctly identified by the classifier, calculated as:

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

- **Specificity:** The proportion of true negatives correctly identified by the classifier, calculated as:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

- **Latency:** The average time taken to make a single prediction, calculated by measuring the total prediction time for the test set and dividing it by the number of test samples.

7. **Results Display:** The confusion matrix and calculated performance metrics are displayed.

```
1 clc;
2 cv = cvpartition(epochLabels, 'HoldOut', 0.3);
3 trainIdx = training(cv);
4 testIdx = test(cv);
5
6 X_train = features(trainIdx, :);
```

```

7  y_train = epochLabels(trainIdx);
8  X_test = features(testIdx, :);
9  y_test = epochLabels(testIdx);
10
11 KNNModel = fitcknn(X_train, y_train, 'Distance', 'minkowski', 'NumNeighbors',
12     5);
13
14 y_pred = predict(KNNModel, X_test);
15
16 confMat = confusionmat(y_test, y_pred);
17
18 tp = confMat(1,1);
19 tn = confMat(2,2);
20 fp = confMat(2,1);
21 fn = confMat(1,2);
22
23 accuracy = (tp + tn) / sum(confMat(:));
24 sensitivity = tp / (tp + fn);
25 specificity = tn / (tn + fp);
26
27 numTestSamples = size(X_test, 1);
28 tic;
29 for i = 1:numTestSamples
30     predict(KNNModel, X_test(i, :));
31 end
32 latency = toc / numTestSamples;
33
34 disp('Confusion Matrix:');
35 disp(confMat);
36 disp(['Accuracy: ', num2str(accuracy)]);
37 disp(['Sensitivity: ', num2str(sensitivity)]);
38 disp(['Specificity: ', num2str(specificity)]);
39 disp(['Latency (average prediction time): ', num2str(latency), ' seconds']);

```

To ensure reproducibility of the K-Nearest Neighbor (KNN) classifier results, it is essential to set a random seed before partitioning the data. The following MATLAB code achieves this by using the `rng` function.

1. **Set Random Seed:** The random seed is set using `rng(1)`; to ensure that the random number generator produces the same sequence of numbers each time the code runs.
2. **Data Partitioning:** The dataset is split into training (70%) and testing (30%) sets using the `cvpartition` function.
3. **Feature and Label Extraction:** Training and testing features and labels are extracted based on the partition indices.
4. **Training the KNN Classifier:** The KNN classifier is trained on the training data using the Minkowski distance metric with 5 neighbors, implemented by the `fitcknn` function.
5. **Prediction:** The trained KNN model predicts the labels of the test data using the `predict` function.

6. **Confusion Matrix Generation:** A confusion matrix is generated using the `confusionmat` function to compare the predicted and actual test labels.

7. **Performance Metrics Calculation:**

- **Accuracy:** The proportion of correctly classified instances, calculated as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Sensitivity (Recall):** The proportion of true positives correctly identified by the classifier, calculated as:

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

- **Specificity:** The proportion of true negatives correctly identified by the classifier, calculated as:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

- **Latency:** The average time taken to make a single prediction, calculated by measuring the total prediction time for the test set and dividing it by the number of test samples.

8. **Results Display:** The confusion matrix and calculated performance metrics are displayed.

```
1  clc;
2  rng(1);
3  cv = cvpartition(epochLabels, 'HoldOut', 0.3);
4  trainIdx = training(cv);
5  testIdx = test(cv);
6
7  X_train = features(trainIdx, :);
8  y_train = epochLabels(trainIdx);
9  X_test = features(testIdx, :);
10 y_test = epochLabels(testIdx);
11
12 KNNModel = fitcknn(X_train, y_train, 'Distance', 'minkowski', 'NumNeighbors',
13                    5);
14
15 y_pred = predict(KNNModel, X_test);
16
17 confMat = confusionmat(y_test, y_pred);
18
19 tp = confMat(1,1);
20 tn = confMat(2,2);
21 fp = confMat(2,1);
22 fn = confMat(1,2);
23
24 accuracy = (tp + tn) / sum(confMat(:));
25 sensitivity = tp / (tp + fn);
26 specificity = tn / (tn + fp);
```

```

27
28 numTestSamples = size(X_test, 1);
29 tic;
30 for i = 1:numTestSamples
31     predict(KNNModel, X_test(i, :));
32 end
33 latency = toc / numTestSamples;
34
35 disp('Confusion Matrix:');
36 disp(confMat);
37 disp(['Accuracy: ', num2str(accuracy)]);
38 disp(['Sensitivity: ', num2str(sensitivity)]);
39 disp(['Specificity: ', num2str(specificity)]);
40 disp(['Latency (average prediction time): ', num2str(latency), ' seconds']);

```

Confusion Matrix:

3	3
1	4

Accuracy: 0.63636

Sensitivity: 0.5

Specificity: 0.8

Latency (average prediction time): 0.00095569 seconds

Figure 7: KNN Results